

✓ Import all the Libraries

```
import warnings
warnings.filterwarnings('ignore')

import io
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from google.colab import files

sns.set_theme(style='whitegrid')
plt.rcParams['figure.figsize'] = (14, 5)
%matplotlib inline

COLORS = ['#2196F3', '#FF5722', '#E91E63', '#4CAF50']
CHANNELS = ['In-Store', 'Uber Eats', 'DoorDash', 'Self-Delivery']

print(' Libraries imported!')
```

Libraries imported!

✓ Step 2: Load & Explore Dataset

```
# Upload the CSV file
print('Upload SkyCity_Auckland_Restaurants___Bars.csv')
uploaded = files.upload()
filename = list(uploaded.keys())[0]

df = pd.read_csv(io.BytesIO(uploaded[filename]))

# Add calculated columns
df['TotalRevenue'] = df['InStoreRevenue'] + df['UberEatsRevenue'] + df['DoorDashRevenue'] + df['SelfD
df['TotalNetProfit'] = df['InStoreNetProfit'] + df['UberEatsNetProfit'] + df['DoorDashNetProfit'] + df[
df['AggregatorShare'] = df['UE_share'] + df['DD_share']
df['DivScore'] = (1 - df[['InStoreShare', 'UE_share', 'DD_share', 'SD_share']].std(axis=1)).round(3)
df['HighRisk'] = df['AggregatorShare'] > 0.70

print(f'\n Loaded: {filename}')
print(f'Shape      : {df.shape}')
print(f'Cuisines   : {sorted(df["CuisineType"].unique().tolist())}')
print(f'Segments   : {sorted(df["Segment"].unique().tolist())}')
print(f'Subregions : {sorted(df["Subregion"].unique().tolist())}')
df.head()
```

Upload SkyCity_Auckland_Restaurants___Bars.csv

Choose Files

 SkyCity Au...s & Bars.csv

SkyCity Auckland Restaurants & Bars.csv(text/csv) - 373317 bytes, last modified: 3/18/2026 - 100% done

Saving SkyCity Auckland Restaurants & Bars.csv to SkyCity Auckland Restaurants & Bars (1).csv

Loaded: SkyCity Auckland Restaurants & Bars (1).csv

Shape : (1696, 35)

Cuisines : ['Burgers', 'Chicken Dishes', 'Chinese', 'Indian', 'Japanese', 'Kebabs/Mediterranean', 'Pi

Segments : ['Cafe', 'Full-service', 'Ghost Kitchen', 'QSR']

Subregions : ['CBD', 'North Shore', 'South Auckland', 'West Auckland']

	CuisineType	RestaurantID	RestaurantName	Segment	Subregion	GrowthFactor	AOV	MonthlyOrders	In
0	Burgers	25731	Urban Burgers House	Cafe	North Shore	1.03	43.97	668	
1	Burgers	25123	Urban Burgers Diner	QSR	South Auckland	1.05	40.45	1388	
2	Burgers	25177	King Burgers Eatery	Cafe	West Auckland	1.04	40.03	1717	
3	Burgers	25540	Classic Burgers Tavern	QSR	North Shore	1.03	36.28	1083	
4	Burgers	25258	Lucky Burgers Bistro	Cafe	South Auckland	1.05	34.34	1230	

5 rows × 35 columns

```
print('=== BASIC INFO ===')
df.info()
print('\n=== STATISTICAL SUMMARY ===')
df.describe().round(2)
```

```

=== BASIC INFO ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1696 entries, 0 to 1695
Data columns (total 35 columns):
#   Column                Non-Null Count  Dtype
---  -
0   CuisineType           1696 non-null  object
1   RestaurantID           1696 non-null  int64
2   RestaurantName         1696 non-null  object
3   Segment               1696 non-null  object
4   Subregion             1696 non-null  object
5   GrowthFactor          1696 non-null  float64
6   AOV                   1696 non-null  float64
7   MonthlyOrders         1696 non-null  int64
8   InStoreOrders         1696 non-null  int64
9   InStoreRevenue        1696 non-null  float64
10  UberEatsOrders        1696 non-null  int64
11  DoorDashOrders        1696 non-null  int64
12  SelfDeliveryOrders    1696 non-null  int64
13  UberEatsRevenue       1696 non-null  float64
14  DoorDashRevenue       1696 non-null  float64
15  SelfDeliveryRevenue   1696 non-null  float64
16  COGSRate              1696 non-null  float64
17  OPEXRate              1696 non-null  float64
18  CommissionRate        1696 non-null  float64
19  DeliveryRadiusKM      1696 non-null  int64
20  DeliveryCostPerOrder  1696 non-null  float64
21  SD_DeliveryTotalCost  1696 non-null  float64
22  InStoreNetProfit      1696 non-null  float64
23  UberEatsNetProfit     1696 non-null  float64
24  DoorDashNetProfit     1696 non-null  float64
25  SelfDeliveryNetProfit  1696 non-null  float64
26  InStoreShare          1696 non-null  float64
27  UE_share              1696 non-null  float64
28  DD_share              1696 non-null  float64
29  SD_share              1696 non-null  float64
30  TotalRevenue          1696 non-null  float64
31  TotalNetProfit        1696 non-null  float64
32  AggregatorShare       1696 non-null  float64
33  DivScore              1696 non-null  float64
34  HighRisk              1696 non-null  bool
dtypes: bool(1), float64(23), int64(7), object(4)
memory usage: 452.3+ KB

```

```

=== STATISTICAL SUMMARY ===

```

	RestaurantID	GrowthFactor	AOV	MonthlyOrders	InStoreOrders	InStoreRevenue	UberEatsOrde
count	1696.00	1696.00	1696.00	1696.00	1696.00	1696.00	1696.
mean	25848.50	1.03	38.52	1190.53	218.98	8422.39	471.
std	489.74	0.02	4.46	422.33	149.76	5851.23	171.
min	25001.00	0.99	29.79	441.00	13.00	412.36	142.
25%	25424.75	1.03	34.79	827.00	101.00	3904.34	331.
50%	25848.50	1.04	38.34	1182.00	178.00	6791.43	456.
75%	26272.25	1.05	42.50	1525.00	307.00	11550.76	602.
max	26696.00	1.05	47.23	2337.00	819.00	35356.23	937.

8 rows × 30 columns

Step 3: Data Validation

```

print('=== DATA VALIDATION ===\n')

# 1. Null check
nulls = df.isnull().sum().sum()
print(f'1. Missing values      : {nulls} ' if nulls == 0 else f'1. Missing values: {nulls} ')

```

```
# 2. Order count consistency
df['OrdersSum'] = df['InStoreOrders'] + df['UberEatsOrders'] + df['DoorDashOrders'] + df['SelfDelivery']
mismatch = (df['OrdersSum'] != df['MonthlyOrders']).sum()
print(f'2. Order count match : {mismatch} mismatches ' if mismatch == 0 else f'2. Mismatches: {mismatch}')

# 3. Share sum check
df['ShareSum'] = (df['InStoreShare'] + df['UE_share'] + df['DD_share'] + df['SD_share']).round(2)
share_issues = (abs(df['ShareSum'] - 1.0) > 0.05).sum()
print(f'3. Share sum ~100% : {share_issues} issues ' if share_issues == 0 else f'3. Share issues: {share_issues}')

# 4. Duplicate check
dups = df.duplicated().sum()
print(f'4. Duplicate rows : {dups} ' if dups == 0 else f'4. Duplicates: {dups} ')

# 5. Outliers
q1, q3 = df['MonthlyOrders'].quantile([0.25, 0.75])
iqr = q3 - q1
outliers = ((df['MonthlyOrders'] < q1 - 1.5*iqr) | (df['MonthlyOrders'] > q3 + 1.5*iqr)).sum()
print(f'5. MonthlyOrders outliers: {outliers}')

print('\n Dataset is clean and ready for analysis!')
```

=== DATA VALIDATION ===

```
1. Missing values      : 0
2. Order count match  : 0 mismatches
3. Share issues: 1665
4. Duplicate rows     : 0
5. MonthlyOrders outliers: 0
```

Dataset is clean and ready for analysis!

✓ Step 4: KPI Summary

```
total_restaurants = df['RestaurantID'].nunique()
total_orders      = df['MonthlyOrders'].sum()
total_revenue     = df['TotalRevenue'].sum()
total_profit      = df['TotalNetProfit'].sum()
avg_aov           = df['AOV'].mean()
high_risk         = df['HighRisk'].sum()
avg_div           = df['DivScore'].mean()

print('=' * 50)
print(' SKYCITY AUCKLAND – KPI SUMMARY')
print('=' * 50)
print(f' Total Restaurants      : {total_restaurants:,}')
print(f' Total Monthly Orders    : {total_orders:,}')
print(f' Total Revenue           : ${total_revenue:,.2f}')
print(f' Total Net Profit        : ${total_profit:,.2f}')
print(f' Overall Profit Margin    : {total_profit/total_revenue*100:.1f}%')
print(f' Avg Order Value (AOV)   : ${avg_aov:.2f}')
print(f' High Risk Restaurants    : {high_risk:,} ({high_risk/total_restaurants*100:.1f}%')
print(f' Avg Diversification      : {avg_div:.3f} / 1.0')
print('=' * 50)

# KPI bar chart
kpi_labels = ['Total\nOrders', 'Total\nRevenue ($M)', 'Net\nProfit ($M)', 'High Risk\nRestaurants']
kpi_vals   = [total_orders/1e6, total_revenue/1e6, total_profit/1e6, high_risk]

fig, axes = plt.subplots(1, 4, figsize=(16, 4))
kpi_colors = ['#2196F3', '#4CAF50', '#FF9800', '#E53935']
kpi_units  = ['M orders', '$M', '$M', 'restaurants']

for ax, label, val, color, unit in zip(axes, kpi_labels, kpi_vals, kpi_colors, kpi_units):
    ax.barh([0], [val], color=color, height=0.4)
    ax.set_xlim(0, val * 1.3)
    ax.set_yticks([])
    ax.set_title(label, fontweight='bold', fontsize=11)
```

```

ax.text(val * 0.5, 0, f'{val:.2f} {unit}', ha='center', va='center',
        color='white', fontweight='bold', fontsize=11)
ax.set_xticks([])
for spine in ax.spines.values(): spine.set_visible(False)

plt.suptitle('SkyCity Auckland – Key Performance Indicators', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```

```

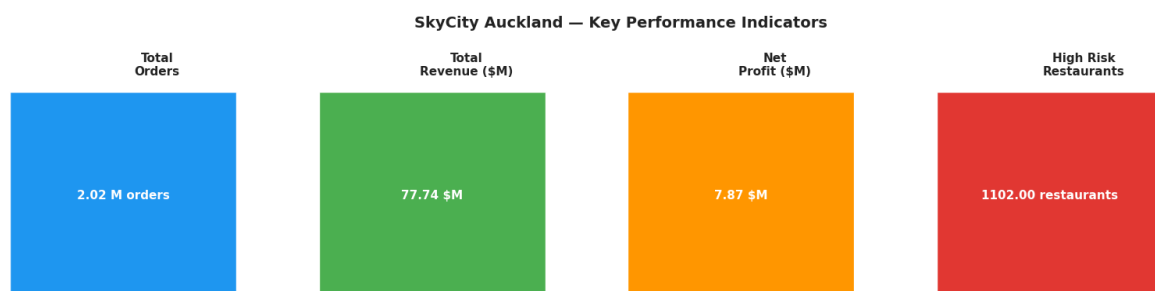
=====
SKYCITY AUCKLAND – KPI SUMMARY
=====

```

```

Total Restaurants      : 1,696
Total Monthly Orders   : 2,019,134
Total Revenue          : $77,739,306.80
Total Net Profit       : $7,866,306.13
Overall Profit Margin  : 10.1%
Avg Order Value (AOV)  : $38.52
High Risk Restaurants  : 1,102 (65.0%)
Avg Diversification    : 0.851 / 1.0
=====

```



✓ Step 5: Channel Volume & Market Share

```

orders = [df['InStoreOrders'].sum(), df['UberEatsOrders'].sum(),
          df['DoorDashOrders'].sum(), df['SelfDeliveryOrders'].sum()]
revenue = [df['InStoreRevenue'].sum(), df['UberEatsRevenue'].sum(),
            df['DoorDashRevenue'].sum(), df['SelfDeliveryRevenue'].sum()]
profit = [df['InStoreNetProfit'].sum(), df['UberEatsNetProfit'].sum(),
           df['DoorDashNetProfit'].sum(), df['SelfDeliveryNetProfit'].sum()]

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Orders bar
bars = axes[0].bar(CHANNELS, orders, color=COLORS, edgecolor='white', linewidth=0.5)
axes[0].set_title('Total Orders by Channel', fontweight='bold')
axes[0].set_ylabel('Orders')
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x):,}'))
axes[0].tick_params(axis='x', rotation=15)
for bar, val in zip(bars, orders):
    axes[0].text(bar.get_x()+bar.get_width()/2, bar.get_height()+8000,
                  f'{val/sum(orders)*100:.1f}%', ha='center', fontsize=10, fontweight='bold')

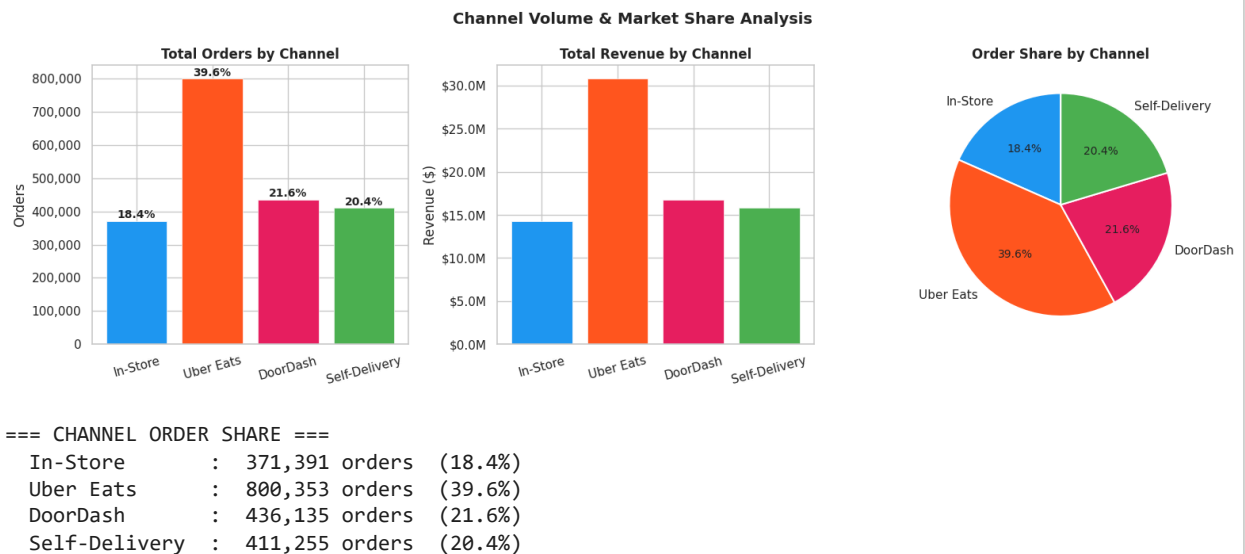
# Revenue bar
axes[1].bar(CHANNELS, revenue, color=COLORS, edgecolor='white', linewidth=0.5)
axes[1].set_title('Total Revenue by Channel', fontweight='bold')
axes[1].set_ylabel('Revenue ($)')
axes[1].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'${x/1e6:.1f}M'))
axes[1].tick_params(axis='x', rotation=15)

# Donut chart
wedges, texts, autotexts = axes[2].pie(
    orders, labels=CHANNELS, autopct='%1.1f%%',
    colors=COLORS, startangle=90,
    wedgeprops={'edgecolor': 'white', 'linewidth': 1.5}
)
axes[2].set_title('Order Share by Channel', fontweight='bold')
for text in autotexts: text.set_fontsize(10)

```

```
plt.suptitle('Channel Volume & Market Share Analysis', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print('\n=== CHANNEL ORDER SHARE ===')
for ch, o in zip(CHANNELS, orders):
    print(f' {ch:<15}: {o:>8,} orders ({o/sum(orders)*100:.1f}%')
```



✓ Step 6: Geographic Channel Analysis

```
sub_orders = df.groupby('Subregion')[
    ['InStoreOrders', 'UberEatsOrders', 'DoorDashOrders', 'SelfDeliveryOrders']
].sum().reset_index()
sub_orders.columns = ['Subregion'] + CHANNELS

fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Grouped bar
x = range(len(sub_orders))
width = 0.2
for i, (ch, color) in enumerate(zip(CHANNELS, COLORS)):
    axes[0].bar([j + i*width for j in x], sub_orders[ch],
                width, label=ch, color=color, edgecolor='white')
axes[0].set_xticks([j + 1.5*width for j in x])
axes[0].set_xticklabels(sub_orders['Subregion'], rotation=15)
axes[0].set_title('Orders by Subregion & Channel', fontweight='bold')
axes[0].set_ylabel('Total Orders')
axes[0].legend(title='Channel', fontsize=8)
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f'{int(x):,}'))

# Stacked % bar
sub_pct = sub_orders.set_index('Subregion')[CHANNELS]
sub_pct = sub_pct.div(sub_pct.sum(axis=1), axis=0) * 100
sub_pct.plot(kind='bar', stacked=True, ax=axes[1],
             color=COLORS, edgecolor='white', linewidth=0.3)
axes[1].set_title('Channel Mix % by Subregion', fontweight='bold')
axes[1].set_ylabel('Share (%)')
axes[1].tick_params(axis='x', rotation=15)
axes[1].legend(title='Channel', fontsize=8)

# Heatmap
sns.heatmap(sub_pct.round(1), annot=True, fmt='.1f',
            cmap='YlOrRd', ax=axes[2], linewidths=0.5,
            cbar_kws={'label': 'Share (%)'})
axes[2].set_title('Channel Share Heatmap by Subregion', fontweight='bold')
axes[2].set_xticklabels(axes[2].get_xticklabels(), rotation=15)
```

```

axes[2].set_yticklabels(axes[2].get_yticklabels(), rotation=0)

plt.suptitle('Geographic Channel Preference Analysis', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print('\n=== DOMINANT CHANNEL BY SUBREGION ===')
for _, row in sub_orders.iterrows():
    vals = row[CHANNELS]
    print(f' {row["Subregion"]}<18} → {vals.idxmax()} ({vals.max():.} orders)')

```



✓ Step 7: Cuisine Channel Analysis

```

cui_orders = df.groupby('CuisineType')[
    ['InStoreOrders', 'UberEatsOrders', 'DoorDashOrders', 'SelfDeliveryOrders']
].sum()
cui_orders.columns = CHANNELS
cui_pct = cui_orders.div(cui_orders.sum(axis=1), axis=0) * 100

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Stacked bar
cui_pct.plot(kind='bar', stacked=True, ax=axes[0],
             color=COLORS, edgecolor='white', linewidth=0.3)
axes[0].set_title('Channel Mix % by Cuisine', fontweight='bold')
axes[0].set_xlabel('Cuisine Type')
axes[0].set_ylabel('Share (%)')
axes[0].tick_params(axis='x', rotation=30)
axes[0].legend(title='Channel', bbox_to_anchor=(1.01, 1))

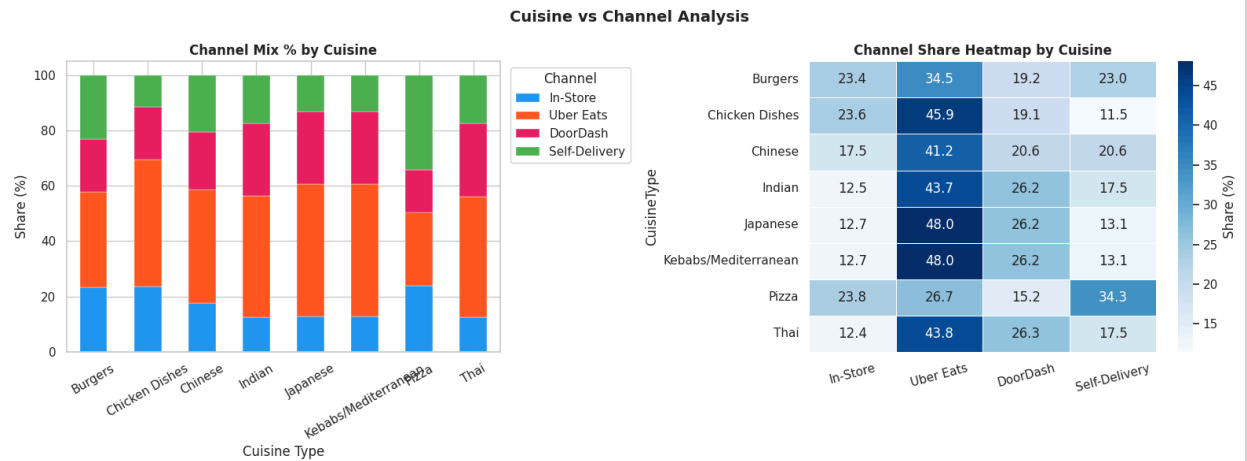
# Heatmap
sns.heatmap(cui_pct.round(1), annot=True, fmt='.1f',
            cmap='Blues', ax=axes[1], linewidths=0.5,
            cbar_kws={'label': 'Share (%)'})
axes[1].set_title('Channel Share Heatmap by Cuisine', fontweight='bold')
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=15)
axes[1].set_yticklabels(axes[1].get_yticklabels(), rotation=0)

plt.suptitle('Cuisine vs Channel Analysis', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print('\n=== DOMINANT CHANNEL BY CUISINE ===')
for cuisine in cui_pct.index:
    dom = cui_pct.loc[cuisine].idxmax()

```

```
val = cui_pct.loc[cuisine].max()
print(f' {cuisine:<25} → {dom} ({val:.1f}%)')
```



```
=== DOMINANT CHANNEL BY CUISINE ===
Burgers           → Uber Eats (34.5%)
Chicken Dishes    → Uber Eats (45.9%)
Chinese           → Uber Eats (41.2%)
Indian            → Uber Eats (43.7%)
Japanese          → Uber Eats (48.0%)
Kebabs/Mediterranean → Uber Eats (48.0%)
Pizza             → Self-Delivery (34.3%)
Thai              → Uber Eats (43.8%)
```

✓ Step 8: Segment Channel Analysis

```
seg_orders = df.groupby('Segment')[
    ['InStoreOrders', 'UberEatsOrders', 'DoorDashOrders', 'SelfDeliveryOrders']
].sum()
seg_orders.columns = CHANNELS
seg_pct = seg_orders.div(seg_orders.sum(axis=1), axis=0) * 100

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

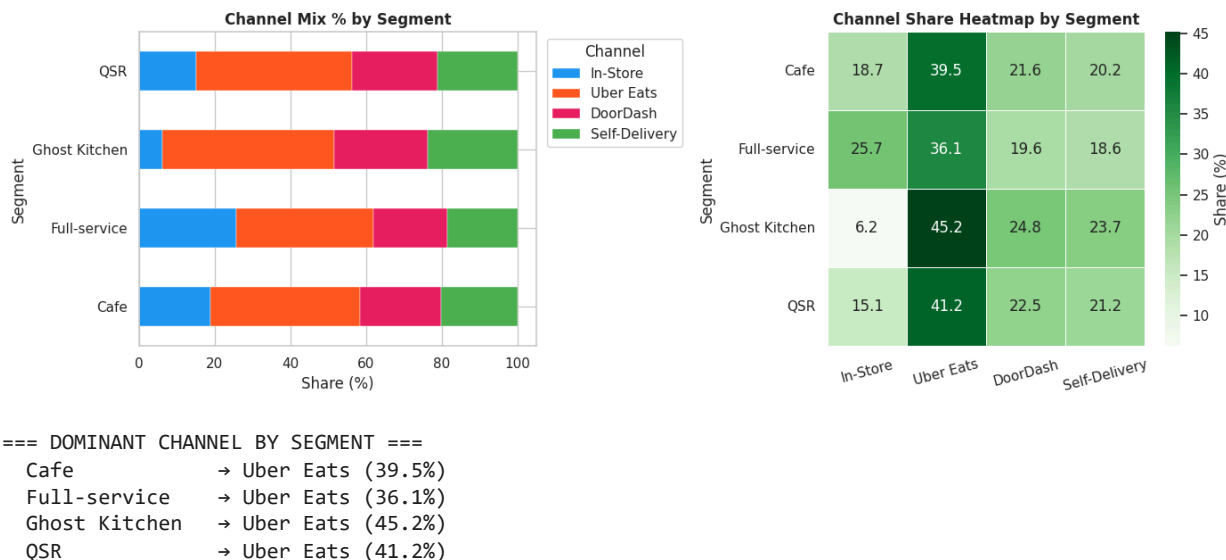
seg_pct.plot(kind='barh', stacked=True, ax=axes[0],
             color=COLORS, edgecolor='white', linewidth=0.3)
axes[0].set_title('Channel Mix % by Segment', fontweight='bold')
axes[0].set_xlabel('Share (%)')
axes[0].legend(title='Channel', bbox_to_anchor=(1.01, 1))

sns.heatmap(seg_pct.round(1), annot=True, fmt='.1f',
            cmap='Greens', ax=axes[1], linewidths=0.5,
            cbar_kws={'label': 'Share (%)'})
axes[1].set_title('Channel Share Heatmap by Segment', fontweight='bold')
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=15)
axes[1].set_yticklabels(axes[1].get_yticklabels(), rotation=0)

plt.suptitle('Segment vs Channel Analysis', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print('\n=== DOMINANT CHANNEL BY SEGMENT ===')
for seg in seg_pct.index:
    dom = seg_pct.loc[seg].idxmax()
    val = seg_pct.loc[seg].max()
    print(f' {seg:<15} → {dom} ({val:.1f}%)')
```


Segment vs Channel Analysis



Step 9: Channel Dependency Risk Analysis

```
print(f'High Risk Restaurants (>70% aggregator): {df["HighRisk"].sum():,} / {len(df):,} ({df["HighRisk"]
```

```
risk_sub = df.groupby('Subregion').agg(
    Total=('RestaurantID', 'count'), HighRisk=('HighRisk', 'sum')
).reset_index()
risk_sub['RiskPct'] = (risk_sub['HighRisk'] / risk_sub['Total'] * 100).round(1)

risk_seg = df.groupby('Segment').agg(
    Total=('RestaurantID', 'count'), HighRisk=('HighRisk', 'sum')
).reset_index()
risk_seg['RiskPct'] = (risk_seg['HighRisk'] / risk_seg['Total'] * 100).round(1)

risk_cui = df.groupby('CuisineType').agg(
    Total=('RestaurantID', 'count'), HighRisk=('HighRisk', 'sum')
).reset_index()
risk_cui['RiskPct'] = (risk_cui['HighRisk'] / risk_cui['Total'] * 100).round(1)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

def risk_color(v):
    return '#E53935' if v > 65 else '#FB8C00' if v > 50 else '#43A047'

# By subregion
colors_sub = [risk_color(v) for v in risk_sub['RiskPct']]
bars = axes[0].bar(risk_sub['Subregion'], risk_sub['RiskPct'], color=colors_sub, edgecolor='white')
axes[0].axhline(65, color='red', linestyle='--', lw=1.2, label='65% threshold')
axes[0].set_title('High Risk % by Subregion', fontweight='bold')
axes[0].set_ylabel('High Risk (%)')
axes[0].tick_params(axis='x', rotation=15)
axes[0].legend()
for bar, val in zip(bars, risk_sub['RiskPct']):
    axes[0].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.5,
        f'{val}%', ha='center', fontsize=10)

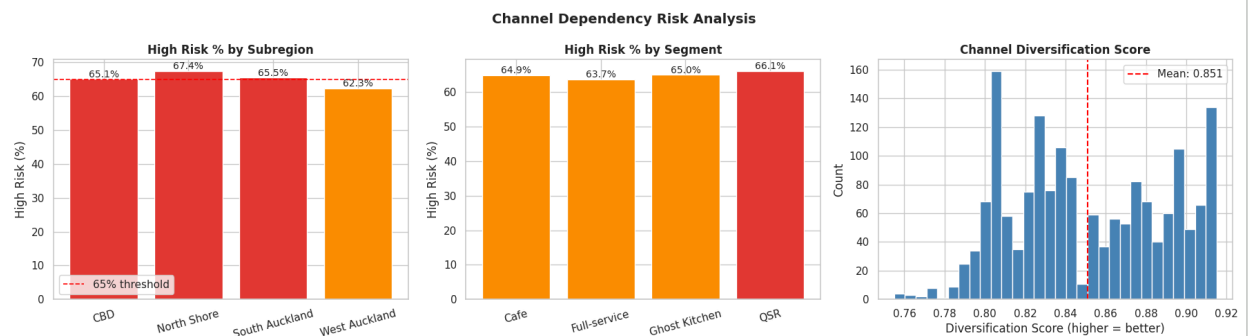
# By segment
colors_seg = [risk_color(v) for v in risk_seg['RiskPct']]
bars2 = axes[1].bar(risk_seg['Segment'], risk_seg['RiskPct'], color=colors_seg, edgecolor='white')
axes[1].set_title('High Risk % by Segment', fontweight='bold')
axes[1].set_ylabel('High Risk (%)')
axes[1].tick_params(axis='x', rotation=15)
for bar, val in zip(bars2, risk_seg['RiskPct']):
    axes[1].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.5,
        f'{val}%', ha='center', fontsize=10)
```

```
# Diversification score distribution
axes[2].hist(df['DivScore'], bins=30, color='steelblue', edgecolor='white')
axes[2].axvline(df['DivScore'].mean(), color='red', linestyle='--',
               label=f'Mean: {df["DivScore"].mean():.3f}')
axes[2].set_title('Channel Diversification Score', fontweight='bold')
axes[2].set_xlabel('Diversification Score (higher = better)')
axes[2].set_ylabel('Count')
axes[2].legend()

plt.suptitle('Channel Dependency Risk Analysis', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print('Risk by Subregion:')
print(risk_sub[['Subregion', 'Total', 'HighRisk', 'RiskPct']].to_string(index=False))
print('\nRisk by Segment:')
print(risk_seg[['Segment', 'Total', 'HighRisk', 'RiskPct']].to_string(index=False))
```

High Risk Restaurants (>70% aggregator): 1,102 / 1,696 (65.0%)



Risk by Subregion:

Subregion	Total	HighRisk	RiskPct
CBD	410	267	65.1
North Shore	383	258	67.4
South Auckland	441	289	65.5
West Auckland	462	288	62.3

Risk by Segment:

Segment	Total	HighRisk	RiskPct
Cafe	504	327	64.9
Full-service	452	288	63.7
Ghost Kitchen	197	128	65.0
QSR	543	359	66.1

Top 20 highest risk restaurants

```
print('=== TOP 20 HIGHEST RISK RESTAURANTS ===')
risk_table = df[df['HighRisk']]
[risk_table[['RestaurantName', 'CuisineType', 'Segment', 'Subregion', 'AggregatorShare', 'DivScore', 'MonthlyOrders']
.sort_values('AggregatorShare', ascending=False).head(20)
risk_table['AggregatorShare%'] = (risk_table['AggregatorShare']*100).round(1)
print(risk_table[['RestaurantName', 'CuisineType', 'Segment', 'Subregion', 'AggregatorShare%', 'DivScore']])
```

=== TOP 20 HIGHEST RISK RESTAURANTS ===

RestaurantName	CuisineType	Segment	Subregion	AggregatorShare%	DivScore
Red Bistro	Japanese	Cafe	North Shore	85.0	0.822
Lotus Bistro	Japanese	Full-service	CBD	85.0	0.832
Dragon Grill	Japanese	Full-service	West Auckland	85.0	0.814
Sakura Cafe	Japanese	Full-service	West Auckland	85.0	0.811
Red Grill	Japanese	Cafe	North Shore	85.0	0.818
Lotus Bistro	Japanese	QSR	South Auckland	85.0	0.795
Sakura Cafe	Japanese	Cafe	CBD	85.0	0.820
Lotus Diner	Japanese	Ghost Kitchen	West Auckland	85.0	0.783
Lotus Tavern	Japanese	Full-service	South Auckland	85.0	0.816
Dragon Eatery	Japanese	Full-service	South Auckland	85.0	0.833
Golden Bistro	Japanese	Full-service	CBD	85.0	0.835
Lotus Grill	Japanese	Full-service	South Auckland	85.0	0.822
Sakura Bistro	Japanese	Cafe	South Auckland	85.0	0.804
Lotus House	Japanese	QSR	North Shore	85.0	0.806
Sakura Bistro	Japanese	Cafe	CBD	85.0	0.816

Sakura House	Japanese	Cafe	CBD	85.0	0.804
Lotus Corner	Japanese	Cafe	South Auckland	85.0	0.806
Lotus Bistro	Japanese	Cafe	South Auckland	85.0	0.801
Lotus Grill	Japanese	Cafe	CBD	85.0	0.804
Sakura Kitchen	Japanese	Cafe	North Shore	85.0	0.814

✓ Step 10: Profitability Analysis

```

profit_df = pd.DataFrame({
    'Channel': CHANNELS,
    'Revenue': [df['InStoreRevenue'].sum(), df['UberEatsRevenue'].sum(),
                df['DoorDashRevenue'].sum(), df['SelfDeliveryRevenue'].sum()],
    'NetProfit': [df['InStoreNetProfit'].sum(), df['UberEatsNetProfit'].sum(),
                 df['DoorDashNetProfit'].sum(), df['SelfDeliveryNetProfit'].sum()]
})
profit_df['Margin%'] = (profit_df['NetProfit'] / profit_df['Revenue'] * 100).round(2)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Margin % by channel
margin_colors = ['#43A047' if m > 15 else '#FB8C00' if m > 5 else '#E53935'
                 for m in profit_df['Margin%']]
bars = axes[0].bar(profit_df['Channel'], profit_df['Margin%'],
                  color=margin_colors, edgecolor='white')
axes[0].set_title('Net Profit Margin % by Channel', fontweight='bold')
axes[0].set_ylabel('Margin (%)')
axes[0].tick_params(axis='x', rotation=15)
for bar, val in zip(bars, profit_df['Margin%']):
    axes[0].text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.2,
                 f'{val:.1f}%', ha='center', fontsize=11, fontweight='bold')

# Revenue vs Profit grouped bar
x = range(len(CHANNELS))
w = 0.35
axes[1].bar([i-w/2 for i in x], profit_df['Revenue']/1e6, w,
            label='Revenue', color='steelblue', edgecolor='white')
axes[1].bar([i+w/2 for i in x], profit_df['NetProfit']/1e6, w,
            label='Net Profit', color='darkorange', edgecolor='white')
axes[1].set_xticks(x)
axes[1].set_xticklabels(CHANNELS, rotation=15)
axes[1].set_title('Revenue vs Net Profit ($M)', fontweight='bold')
axes[1].set_ylabel('Amount ($M)')
axes[1].legend()

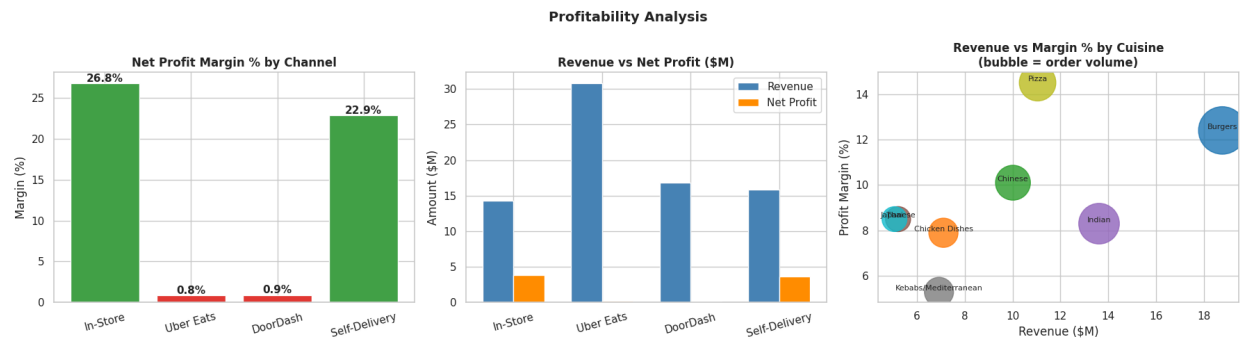
# Cuisine profitability
cui_perf = df.groupby('CuisineType').agg(
    Revenue=('TotalRevenue', 'sum'),
    Profit=('TotalNetProfit', 'sum'),
    Orders=('MonthlyOrders', 'sum')
).reset_index()
cui_perf['Margin%'] = (cui_perf['Profit']/cui_perf['Revenue']*100).round(1)

axes[2].scatter(cui_perf['Revenue']/1e6, cui_perf['Margin%'],
               s=cui_perf['Orders']/200, alpha=0.8,
               c=range(len(cui_perf)), cmap='tab10')
for _, row in cui_perf.iterrows():
    axes[2].annotate(row['CuisineType'],
                    (row['Revenue']/1e6, row['Margin%']),
                    fontsize=8, ha='center', va='bottom')
axes[2].set_title('Revenue vs Margin % by Cuisine\n(bubble = order volume)', fontweight='bold')
axes[2].set_xlabel('Revenue ($M)')
axes[2].set_ylabel('Profit Margin (%)')

plt.suptitle('Profitability Analysis', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

```

```
print('\nChannel Profitability Summary:')  
print(profit_df.to_string(index=False))
```



Channel Profitability Summary:

Channel	Revenue	NetProfit	Margin%
In-Store	14284378.07	3828417.36	26.80
Uber Eats	30816371.49	258488.08	0.84
DoorDash	16789380.95	151122.25	0.90
Self-Delivery	15849176.29	3628278.44	22.89

Step 11: Insights & Recommendations

/4.4.1/chart.umd.js'></script>

```
0d0d; color: #e0e0e0; padding: 18px; }  
nt: space-between; margin-bottom: 16px; }
```

```
: 20px; background: #2a1a0e; color: #FF9800; }  
fr); gap: 10px; margin-bottom: 14px; }  
border-radius: 10px; padding: 13px 14px; }  
ine-height: 1.2; }  
-top: 3px; }  
border-radius: 10px; padding: 15px; margin-bottom: 12px; }  
-transform: uppercase; letter-spacing: .6px; margin-bottom: 10px; }  
12px; margin-bottom: 12px; }  
12px; margin-bottom: 12px; }  
12px; margin-bottom: 12px; }  
ottom: 8px; font-size: 11px; color: #666; }  
ay: inline-block; margin-right: 4px; }  
12px; margin-bottom: 10px; }
```

```
4,1fr); gap: 10px; margin-top: 4px; }  
ng: 12px; }  
8px; font-weight: 600; }
```

```
10px; font-weight: 500; margin-left: 6px; }
```

```
</span> - Order Channel Analytics</div>  
ars • Channel Performance & Market Share</div>
```

```
regions</span>
```

```
>2.02M</span><span class='kl'>Monthly orders</span></div>  
>$77.7M</span><span class='kl'>Total revenue</span></div>  
>$7.87M</span><span class='kl'>Net profit</span></div>  
>$38.52</span><span class='kl'>Avg order value</span></div>  
>1,102</span><span class='kl'>High risk (65%)</span></div>  
>0.851</span><span class='kl'>Avg div. score</span></div>
```

```
n>In-Store</span>  
n>Uber Eats</span>  
n>DoorDash</span>  
n>Self-Delivery</span>
```

```
:12px; '>  
'c1'></canvas></div>  
'c2'></canvas></div>
```

```
'c3'></canvas></div>
```

```
'c4'></canvas></div>
```

```
n>In-Store</span>  
n>Uber Eats</span>  
n>DoorDash</span>  
n>Self-Delivery</span>
```

```
5'></canvas></div>
```

```
'c6'></canvas></div>
```

```
'c7'></canvas></div>  
ndency risk by subregion</div>  
'c8'></canvas></div>
```

```
iv>
'c9'></canvas></div>
```

```
but barely profits</div><div class='ins-s'>39.6% of all orders – only 0.84% profit margin</div></div>
  profitable channel</div><div class='ins-s'>Just 18.4% of orders but 26.8% profit margin</div></div>
re high risk</div><div class='ins-s'>1,102 restaurants over 70% aggregator reliance</div></div>
vulnerable segment</div><div class='ins-s'>Only 6.2% in-store – highest aggregator dependency</div></div>
er utilised</div><div class='ins-s'>22.9% margin, no commission – best alternative</div></div>
```

```
.1%</span></div>
>High-risk restaurants</div>
```

```
.4%</span></div>
>Highest risk subregion</div>
```

```
.5%</span></div>
>High-risk restaurants</div>
```

```
62.3%</span></div>
>Lowest risk subregion</div>
```

```
;
};
```

```
y'],
oundColor: COLS, borderRadius: 4, borderSkipped: false }]
```

```
e6 ? (v/1e6).toFixed(1)+'M' : v >= 1000 ? (v/1000).toFixed(0)+'k' : v } }
w.toLocaleString()) orders` } } }
```

```
y'],
Color: COLS, borderRadius: 4, borderSkipped: false }]
```

```
'M' } }
aw}M revenue` } } } }
```

```
y'],
oundColor: COLS, borderWidth: 0, hoverOffset: 6 }]
```

```
el}: ${((c.raw/2019134*100).toFixed(1))}%` } } }
```

```
y'],
```

```
50'],
```

```
} }
w}% margin` } } }
```

```
land'],
```

```
ackgroundColor:IS, borderRadius:2 },
ackgroundColor:UE, borderRadius:2 },
ackgroundColor:DD, borderRadius:2 },
ackgroundColor:SD, borderRadius:2 }
```

```
: { callback: v => v+'%' } }
: c => ` ${c.dataset.label}: ${c.raw}%` } } }
```

```
e', 'Kebabs', 'Pizza', 'Thai'],
```

```
7,12.7,23.8,12.4], backgroundColor:IS, borderRadius:2 },
0,48.0,26.7,43.8], backgroundColor:UE, borderRadius:2 },
2,26.2,15.2,26.3], backgroundColor:DD, borderRadius:2 },
1,13.1,34.3,17.5], backgroundColor:SD, borderRadius:2 }
```

```
: { callback: v => v+'%' } },
```

```

ckgroundColor:IS, borderRadius:2 },
ackgroundColor:UE, borderRadius:2 },
ackgroundColor:DD, borderRadius:2 },
ackgroundColor:SD, borderRadius:2 }

```

```

: { callback: v => v+'%' } },

```

```

land'],

```

```

00'],

```

```

lback: v => v+'%' } }
w}% high-risk` } } } }

```

```

y'],

```

```

backgroundColor:COLS.map(c=>c+'66'), borderRadius:4, borderSkipped:false },
backgroundColor:COLS,                borderRadius:4, borderSkipped:false }

```

```

10, color: '#888', font: { size: 11 } } },
l}: $$ {c.raw}M` } } },

```

```

v+'M' } }

```

```

ace(chr(10), "&#10;"))}" width="100%" height="1800px" style="border:none;border-radius:12px;"></iframe>

```